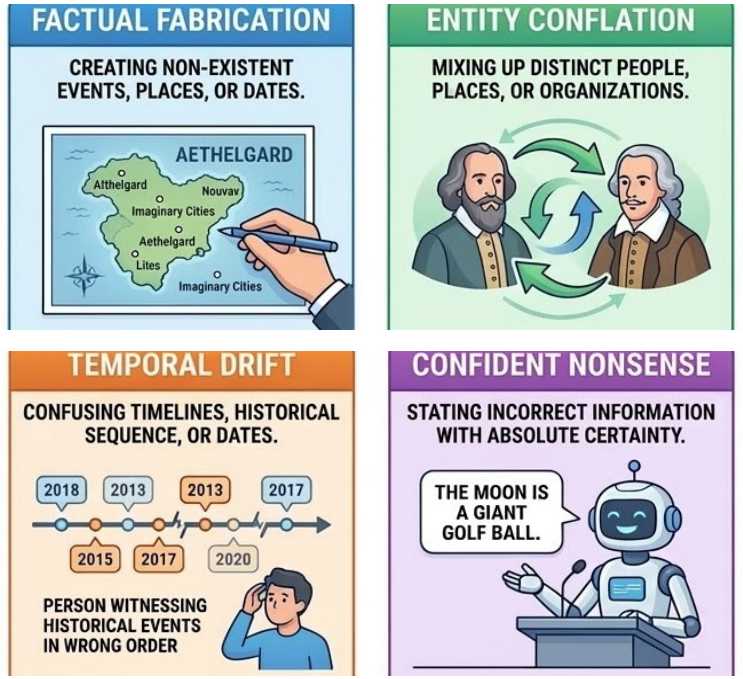
THE HALLUCINATION PROBLEM

- LLMs are remarkably fluent but fluency is not the same as accuracy
- When an LLM generates text that sounds authoritative and confident yet is factually wrong, we call it a **hallucination**

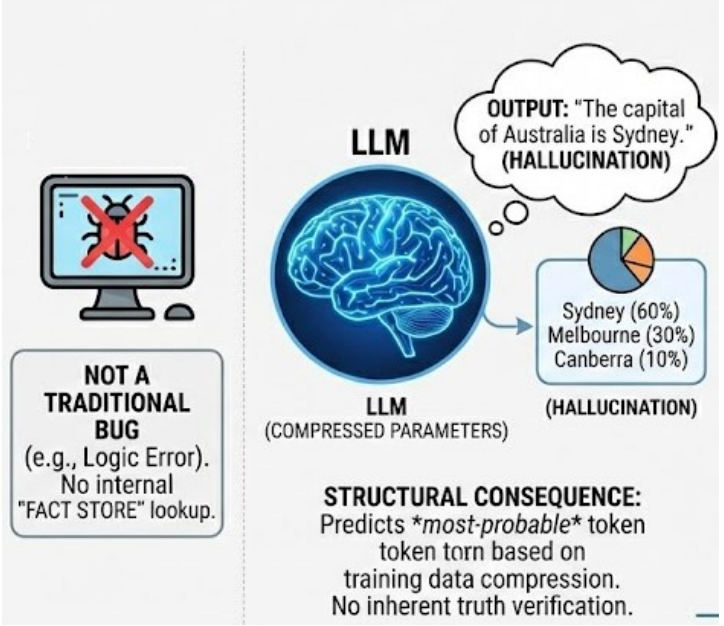
Hallucinations fall into several categories

- **Factual fabrication**: Inventing facts, dates, or citations that don't exist
- **Entity conflation**: Mixing attributes of two real entities
- **Temporal drift**: Presenting outdated information as current
- **Confident nonsense**: Generating plausible-sounding but meaningless statements

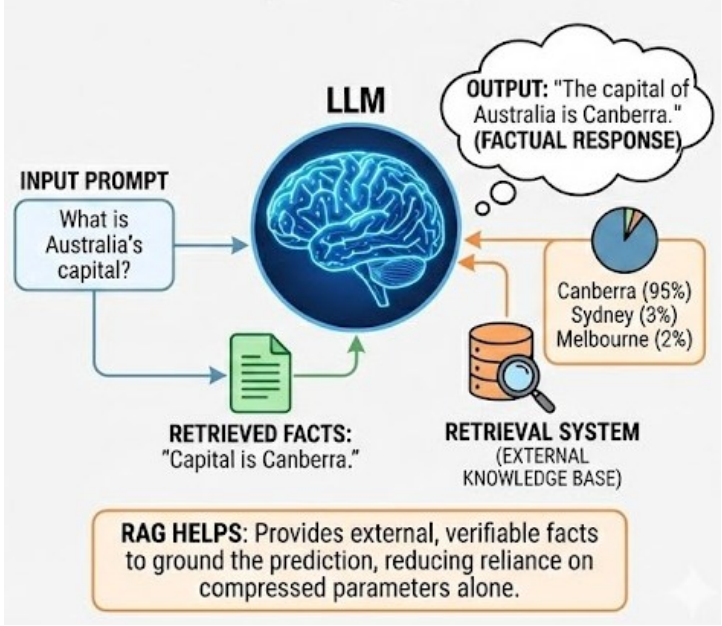


ARE HALLUCINATIONS BUGS IN THE LLM?

- Hallucinations are not bugs in the traditional sense; they are a structural consequence of how LLMs work
- A language model is trained to predict the next most-probable token, not to verify truth
- It has no "fact store" it can look up; everything it knows is compressed into its parameters during training



Using an external "fact store" can help in grounding the responses



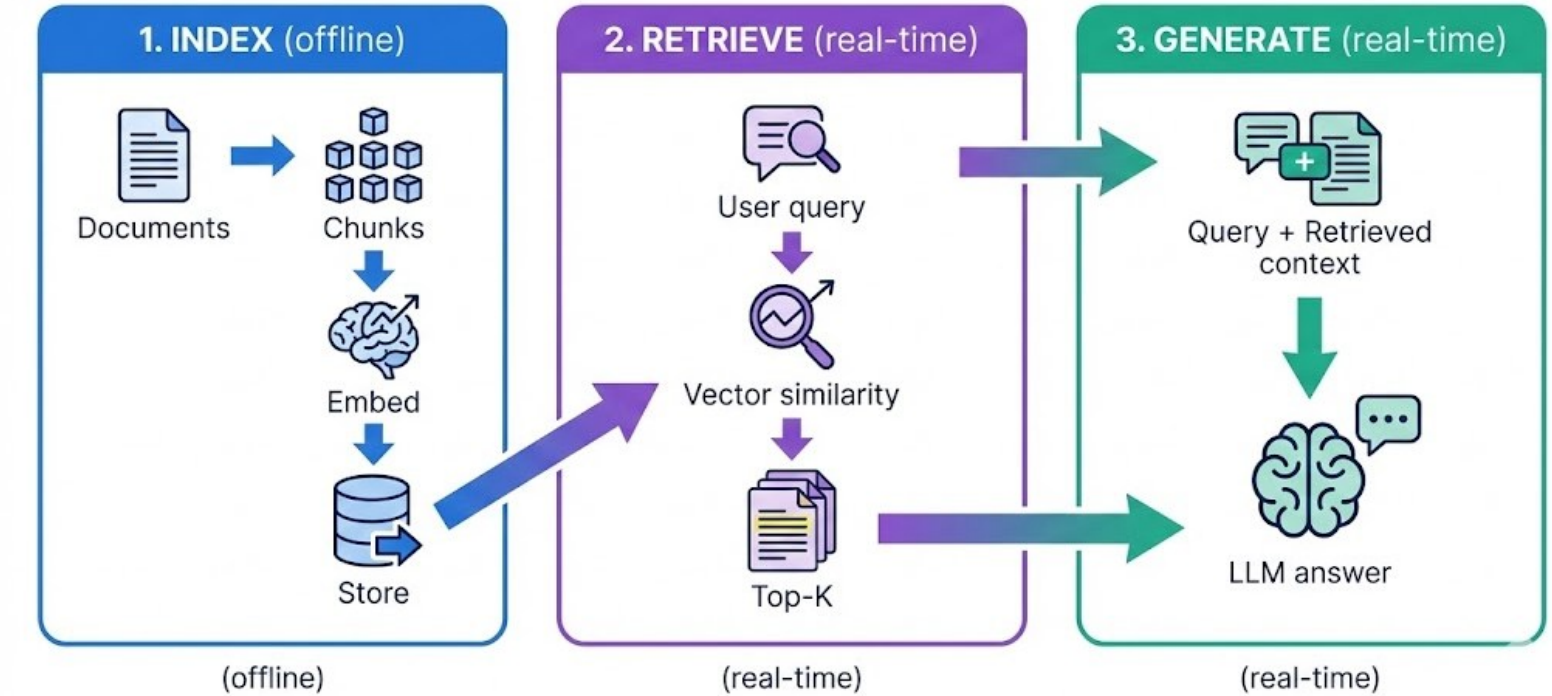
HOW MUCH DOES AN LLM “KNOW”?

- A standard LLM as a closed-book exam taker. It can only rely on what it memorised during training.
- This creates several hard constraints:
  - **Knowledge cutoff**
    Training data has a fixed end date; anything that happened after that date is invisible to the model
  - **No access to private data**
    Enterprise documents, internal wikis, proprietary databases - none of these exist in the model's weights
  - **Compression loss**
    Lossy compression of billions of documents into model parameters means recall is imperfect; rare facts are easily lost
  - **No source attribution**
    The model cannot point to where it learned something, making verification impossible
  - **Static knowledge**
    Retraining or fine-tuning to update knowledge is expensive and slow

KNOWLEDGE-GROUNDED GENERATION

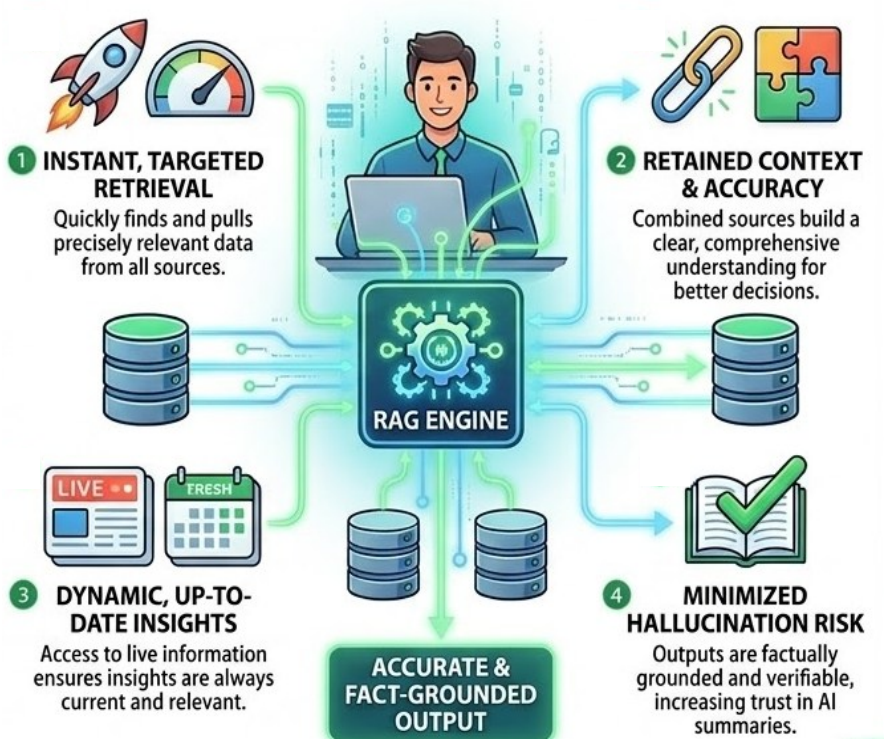
Retrieval-Augmented Generation (RAG) transforms the LLM from a closed-book to an open-book exam taker

Instead of relying solely on memorised knowledge, or fine-tuning the model again, we provide relevant reference material at inference time

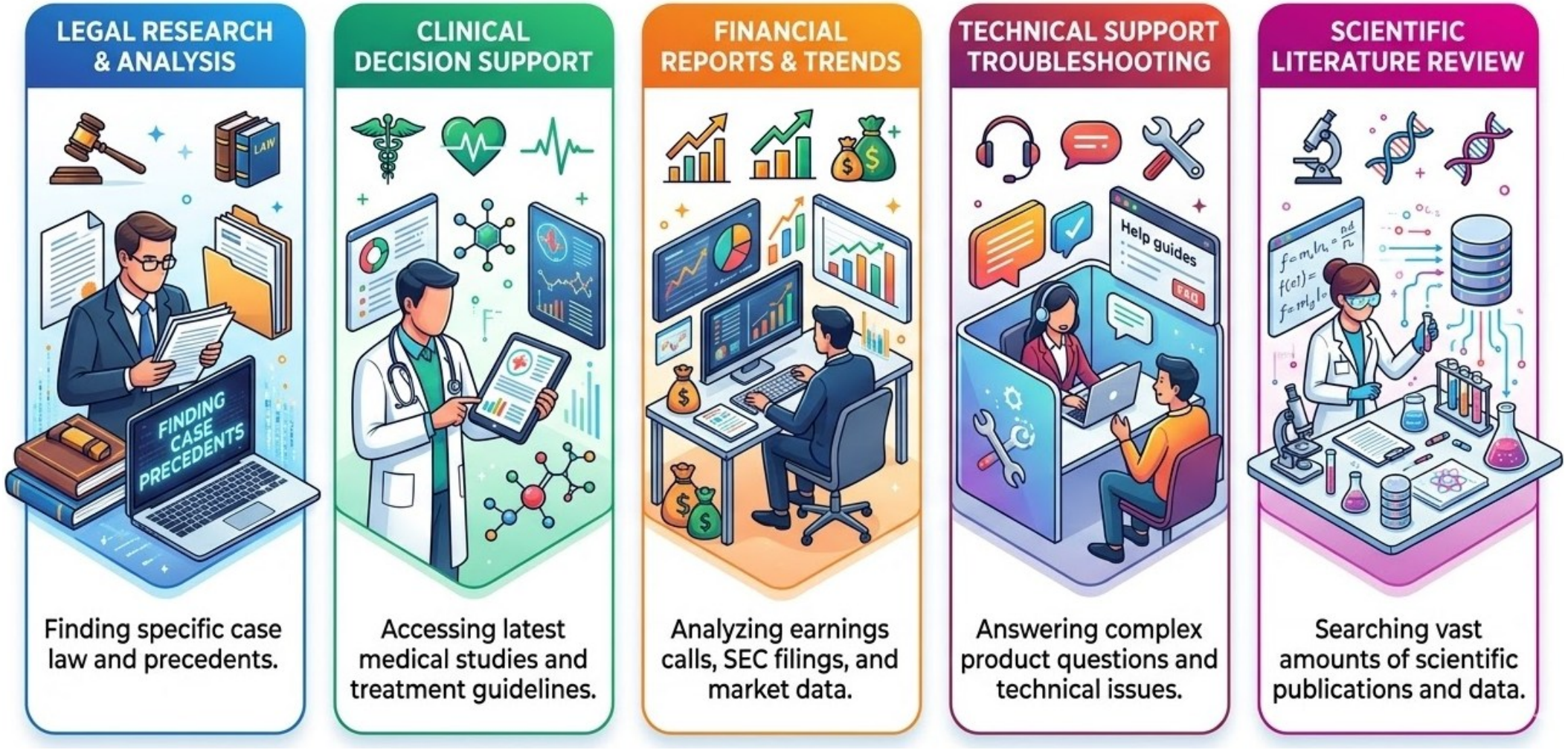


HOW RAG PREVENTS HALLUCINATIONS?

- **Grounding**: The model's answer is anchored to retrieved documents, drastically reducing hallucinations
- **Freshness**: Swap out or update the document store without retraining the model
- **Transparency**: Every answer can cite its source chunks, enabling auditability
- **Cost efficiency**: Updating a vector store is orders of magnitude cheaper than fine-tuning a model
- **Access control**: You can control which documents are retrievable per user or role



RAG USE-CASES



RAG vs FINE-TUNING

Dimension	RAG	Fine-Tuning
Knowledge updates	Instant (update the store)	Slow (retrain)
Factual accuracy	High (grounded in sources)	Variable (can still hallucinate)
Cost	Low (embedding + retrieval)	High (GPU compute for training)
Style/behaviour change	Limited	Effective
Best for	Fact-heavy Q&A, enterprise search	Tone, format, domain-specific language

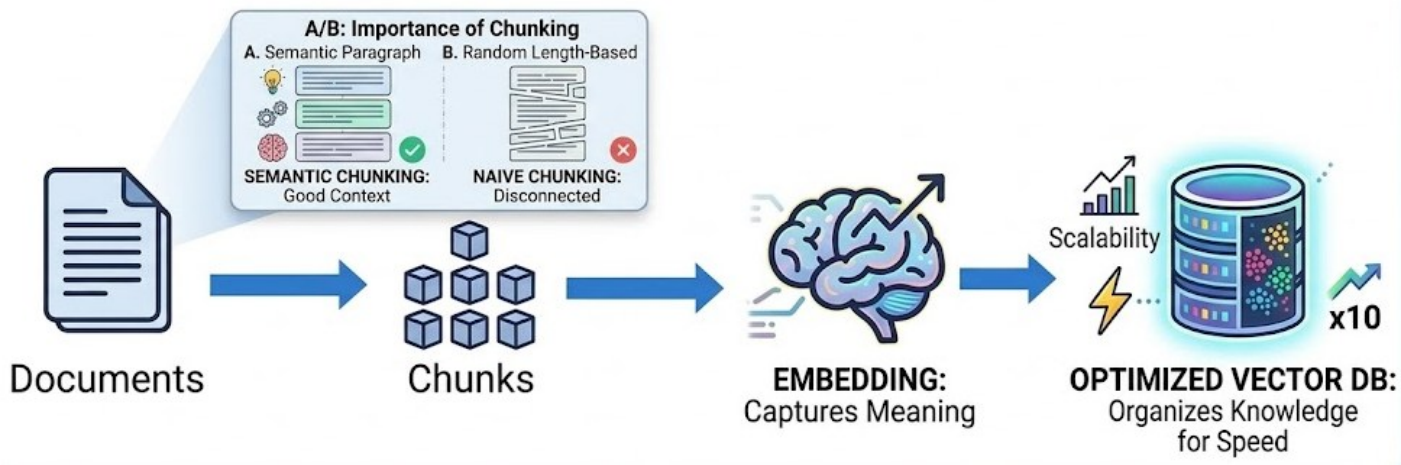
RAG only augments the LLM

LLM is the reasoning engine and RAG provides it with an updatable, verifiable knowledge base

RAG INDEXING PROCESS

To implement a fact-index for RAG to refer, it is important to establish three things:

- **Chunking Strategy**
- **Embedding Model**
- **Vector Database**



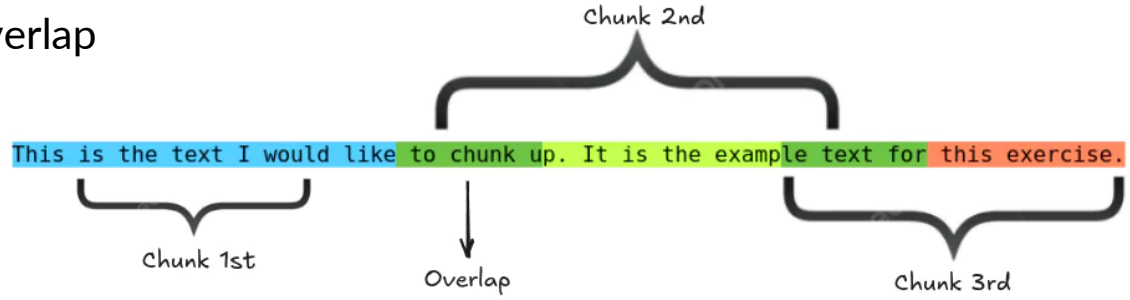
CHUNKING

- Before embedding, documents must be split into **chunks**, smaller units that fit within the embedding model's context window and produce focused embeddings
- Smaller chunks ensure the retriever finds exact relevant needed instead of returning too much noise
- Chunking size is a critical design decision

Size	Pros	Cons
Small (100-200 tokens)	Precise retrieval, focused semantics	May lose context; more chunks to store
Medium (300-500 tokens)	Good balance of precision and context	May split ideas mid-sentence
Large (500-1000 tokens)	Retains full context for complex topics	Diluted semantics; less precise retrieval

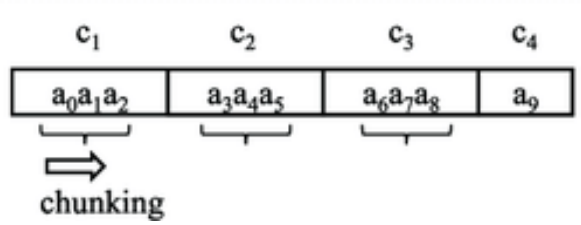
Chunking may introduce information loss when one segment is broken into two parts

- To avoid this, we may add with a **chunk overlap**
- A simple way is to just split with a fixed size and add overlap
- An tuned overlap of 10-20% prevents information loss at boundaries
- There are other rule-based ways to chunk documents, we can add overlaps in these as well

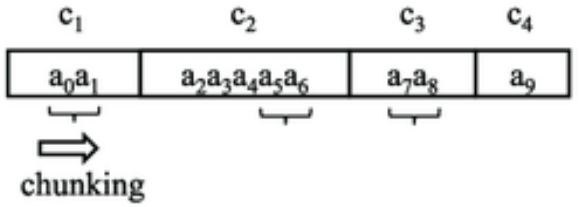


TYPES OF CHUNKING

- In **fixed-length chunking**, documents are split into chunks with a constant token or character length (e.g., 500 tokens per chunk)
- Simple to implement and computationally efficient
- Often used with token-based splitters in frameworks such as LangChain
- Limitation: semantic boundaries (sentences/sections) may be broken; overlap may solve it to some point



E.g., a document divided every 3 tokens regardless of paragraph structure



Variable boundaries, marked by a rolling hash function

- In **variable-size chunking**, chunk length is not fixed; it depends on semantic or structural boundaries
- Different rules which allow flexible size but usually still respects a maximum limit, calculated with **content-defined chunking**
- Often used when preserving meaning is more important than strict token limits
- It solves the boundary shift problem of fixed-size chunks but needs additional computation for each byte (finding a specific boundary pattern)

STRUCTURE-BASED CHUNKING

Natural-Delimiter-Based Chunking

- Splits using structural delimiters such as headings, bullet lists, section markers, markdown blocks
- Attempts to respect human-written document structure

Paragraph-Based Chunking

- Each paragraph becomes a chunk
- Works well when documents already contain coherent structure
- Common in articles, blogs, and documentation

Sentence-Based Chunking

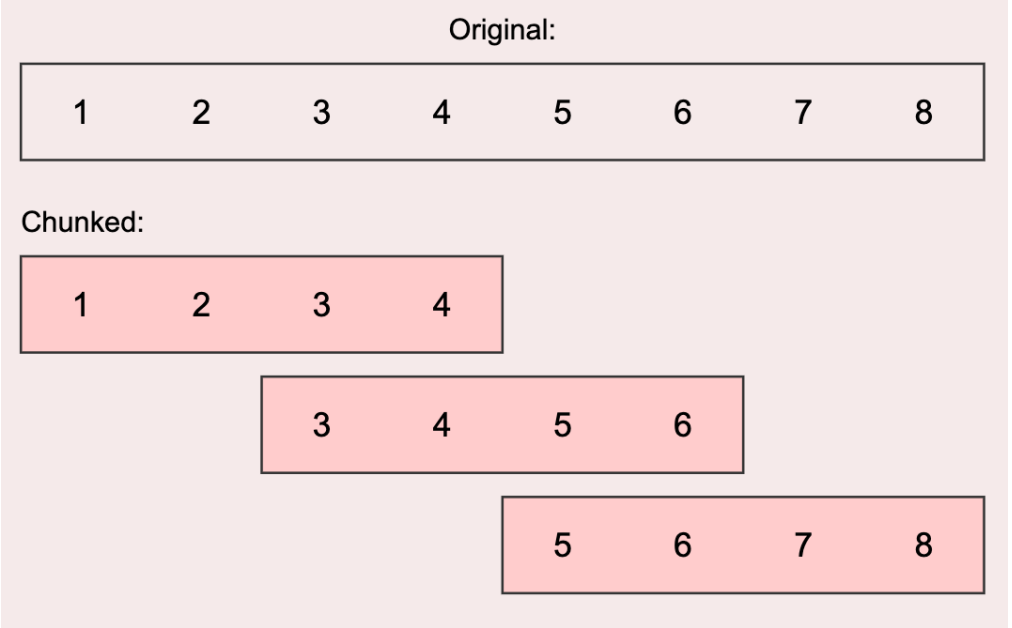
- Each chunk corresponds to one or several sentences
- Sentence boundaries are detected using NLP tokenisers
- Preserves grammatical meaning but may create many small chunks

WINDOW-BASED CHUNKING

- **Sliding window** uses a moving window across the text to generate chunks
- Similar to overlap but conceptually framed as a continuous window
- For example:

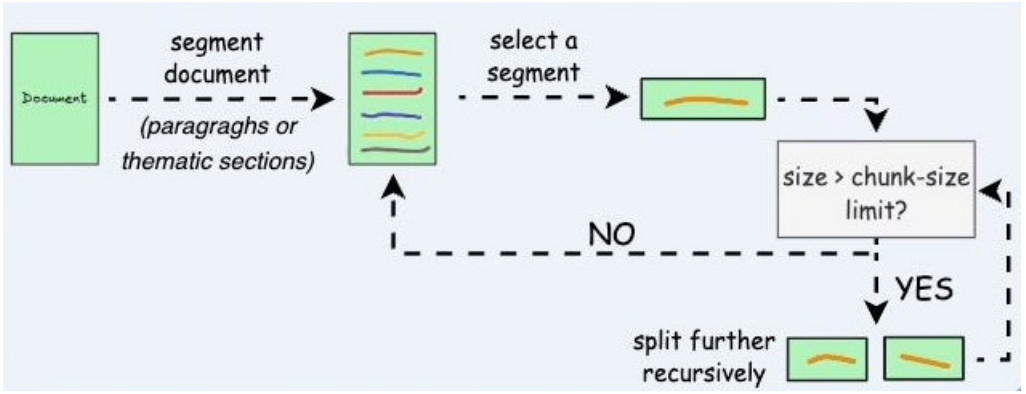
  Window size = 500 tokens, Stride = 100 tokens

  Chunk 1: *1–500*
  Chunk 2: *401–900*



RECURSIVE CHARACTER SPLITTING

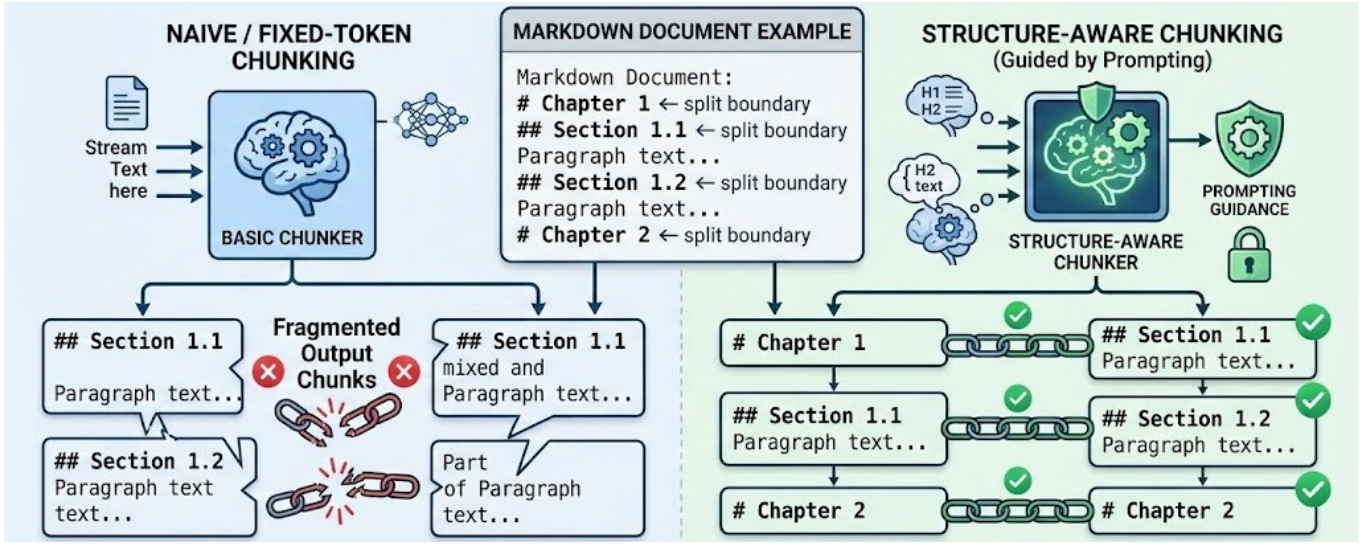
- Recursive splitting is a hierarchical text segmentation, which recursively breaks text into smaller pieces using a priority list of separators
- It follows a sequence of separators from largest structural unit to smallest
- For example: document → sections → sentences → words
- The separator priority: [‘\n\n’, ‘\n’, ‘.’, ‘ ’, ‘’]
- If a structure-based chunk exceed the maximum chunk size, it proceeds to recursively chunk it further
- It is best suited for long documents or structured content, such as PDFs, research papers, or technical manuals
- This is the default chunking strategy in many modern frameworks, like LangChain



STRUCTURE-AWARE CHUNKING

Structure-aware chunking splits documents based on their explicit structural elements rather than arbitrary token limits

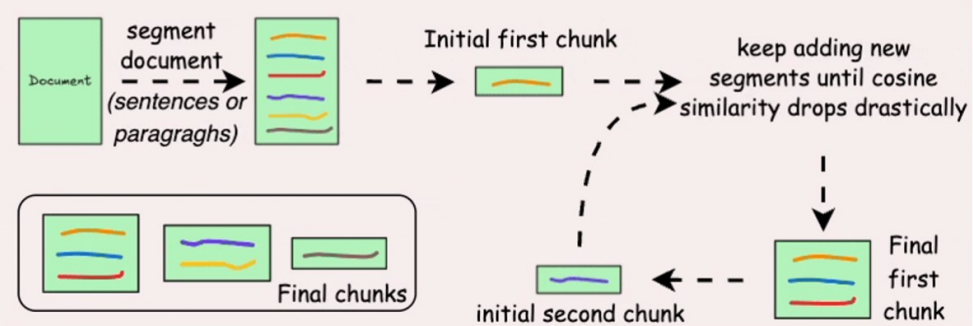
- For structured documents (HTML, Markdown, code, PDFs with headings), structure-aware chunking respects the document's inherent hierarchy
- The goal is to keep logically related content together so that retrieved chunks remain coherent
- Structure boundaries could be headings, subheadings, paragraphs, lists, HTML tags etc.



Recursive chunking splits text progressively using a hierarchy of delimiters until the chunk size constraint is reached, while structure-aware chunking splits based on explicit structural elements to preserve logical organisation

SEMANTIC CHUNKING

- Semantic chunking groups together sentences that are semantically related
- It detects topic shifts and splits; more expensive but produces coherent chunks
- Semantic chunking improves context retrieval and recall during search and helps in reducing hallucinations
- Semantic chunking methods:
  1. Split the document into sentences, compute **similarity** between adjacent sentence embeddings, and chunk them together when similarity drops below a threshold
  2. Use semantic analyses to perform **topic segmentation** and split into chunks based on topic similarity between sentence groups
  3. Use **LLMs** for identifying logical sections



COMPARISON

Method	Typical Use Case	Advantages	Disadvantages
Fixed Length Chunking	Large corpora where simplicity and speed are important (logs, raw text, scraped datasets)	Simple implementation; Predictable chunk size; Efficient indexing and embedding	Breaks semantic boundaries; May split sentences or concepts; Retrieval quality may decrease
Sliding Window Chunking	Context-sensitive retrieval where boundary information may be lost (QA systems, conversational knowledge bases)	Preserves context across chunks through overlap; Reduces information loss at boundaries	Produces duplicate information; Increases embedding storage and retrieval cost
Natural Delimiter Chunking	Documents with clear textual separators (e.g., \n\n, headings, section markers)	Respects natural text boundaries; More coherent chunks than fixed splitting	Delimiters may be inconsistent across documents; Chunk sizes can become uneven
Paragraph & Sentence Chunking	Articles, reports, blogs, educational content	Maintains grammatical and logical meaning; Easier for models to interpret	Paragraphs may exceed token limits; Sentences alone may produce many small chunks
Recursive Chunking	General-purpose RAG pipelines when document structure is unknown or inconsistent	Flexible and widely applicable; Progressively finds smaller boundaries to satisfy token limits;	Still delimiter-based, so true semantic boundaries may not always be preserved
Structure-Aware Chunking	Technical documentation, Markdown/HTML files, manuals, knowledge bases	Preserves document hierarchy and logical sections; Improves retrieval relevance	Requires structured documents; Parsing complexity increases
Semantic Chunking	High-quality retrieval systems where conceptual coherence is critical (research papers, long reports)	Splits based on topic or meaning; Produces highly coherent chunks	Computationally expensive; Requires embedding similarity or additional processing

CREATING EMBEDDING VECTORS

Once we chunk the documents, the next steps is to embed them for searching

Embedding models convert data (text, images, audio etc.) into vector embeddings which capture semantic meaning

There are many available embeddings models, both paid and free

Aspect	Proprietary (e.g., OpenAI, Cohere, Google)	Open-Source (e.g., sentence-transformers, E5)
Quality	State-of-the-art on benchmarks (MTEB)	Very competitive; top OSS models rival proprietary
Cost	Pay-per-token (adds up at scale)	Free (you host it)
Privacy	Data sent to external API	Data stays on your infrastructure
Latency	Network round-trip	Local inference, can be faster
Maintenance	Managed by provider	You manage infra, updates, GPU
Dimension	Fixed per model (e.g., 1536 for text-embedding-3-small)	Varies (384 to 1024+ typical)

- **Prototyping/learning**: Use OSS (all-MiniLM-L6-v2) free, fast, no API key needed
- **Production with budget**: OpenAI or Cohere managed, reliable, high quality
- **Production with privacy constraints**: Self-host BGE or E5 on your own GPU

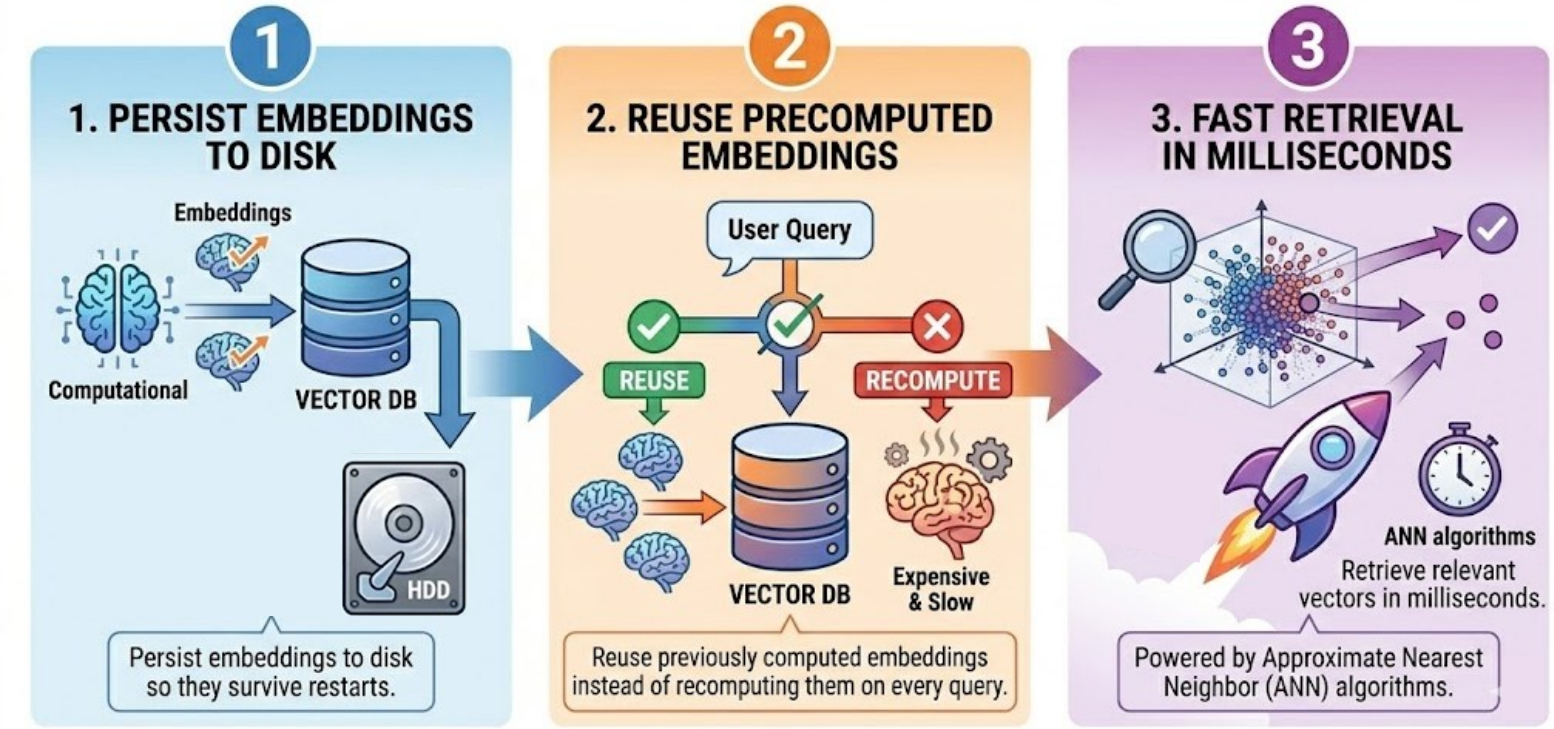
STORING EMBEDDING VECTORS

Computing on embeddings is not free

For a corpus of 100,000 document chunks using a cloud embedding API, you pay both in latency and money

This is why **vector stores** exist, as they let you:

- **Persist** embeddings to disk so they survive restarts
- **Reuse** previously computed embeddings instead of recomputing them on every query
- **Retrieve** relevant vectors in milliseconds using approximate nearest neighbour (ANN) algorithms



VECTOR STORES & DBs

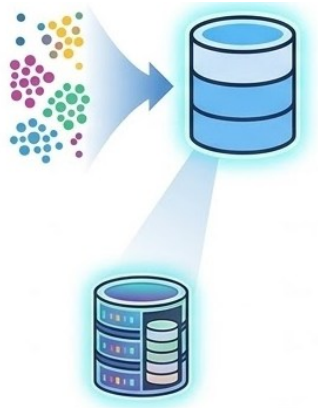
Both are used to store and retrieve vector embeddings for similarity search in AI systems

The difference lies in their scope and capabilities

Vector Store

Focused on storage and retrieval of embeddings

- Efficient storage of embeddings
- Fast similarity search using vector indexes
- Often used as a component within larger applications
- Suitable for small-to-medium datasets
- Provides simple querying capabilities



Vector DB

Full database system built for vector data

- Robust database management features
- Supports complex queries and CRUD operations
- Designed for large-scale and distributed systems
- Advanced indexing mechanisms (e.g., ANN methods)
- Persistence, security, and access control

POPULAR VECTOR DBs